

Forward–Reverse Architecture Contract

Status: IMPLEMENTED · UNIT_TESTED Executable form: pkg/fref (26 tests) · fabric: FREF

1. Why a contract and not a diagram

Both directions already existed in the repository, spread across the EQF, the intelligence layer and the workflow engine. What did not exist was a statement that they are **one architecture** — in what order their stages must run, which package owns each stage, and what it means for the two directions to *close* over the same claim.

A diagram cannot refuse anything. This contract can.

2. The two directions

FORWARD	REVERSE
OBSERVATION	DECISION / CLAIM
↓	↓
EVIDENCE	PROOF OBLIGATIONS
↓	↓
KNOWLEDGE	REQUIRED EVIDENCE
↓	↓
REASONING	EVIDENCE GAP
↓	↓
TRUST	CONTRADICTION
↓	↓
FINDING	QUALIFICATION
↓	↓
DECISION	NEXT BEST EVIDENCE

Forward is how most systems work. Reverse is what makes VERIQO a **proof-oriented** system rather than a detection engine: it starts from what somebody asserts and asks what would have to be true.

3. Stage bindings

Every stage names the package that performs it, what that package is authoritative for, and what it refuses when it cannot complete. The bindings are the anti-duplication record: a stage with no binding would describe work nothing does, and a stage that runs elsewhere is reported as **drift** by `Execution.VerifyAgainstContract`.

Forward

Stage	Package	Authoritative for	Fails closed by
OBSERVATION	pkg/dataplatform/ingest	what was observed, from which source, under which licence	rejecting an unlicensed or unattributed observation at ingest — not down-weighting it
EVIDENCE	pkg/evidence/manifest	the canonical evidence record, versions, custody	never admitting evidence that cannot be pinned to a version and a content hash

Stage	Package	Authoritative for	Fails closed by
KNOWLEDGE	pkg/moat/kg	entities, links, the knowledge graph	leaving a below-threshold resolution unresolved rather than merging two parties
REASONING	pkg/moat/causal	hypotheses and causal structure — proposals, never findings	producing no hypothesis when it cannot cite its inputs
TRUST	pkg/core/trustcalc	trust standing of sources and what rests on them	treating an unassessed source as UNKNOWN, never as independent
FINDING	pkg/proof	whether a sealed proof object founds a finding	founding no finding on an insufficient object; yielding next-best evidence instead
DECISION	pkg/proof	the operational act on an authorized finding	making a decision without an authorized finding unconstructible

Reverse

Stage	Package	Authoritative for	Fails closed by
CLAIM	pkg/casefabric	the proposition a case must establish	refusing a claim with no falsifiable proposition at registration
PROOF_OBLIGATIONS	pkg/qualification/reverseproof	what would have to be shown	rejecting a requirement with no falsifying observation — it is not a test
REQUIRED_EVIDENCE	pkg/qualification/reverseproof	the evidence each obligation calls for	failing the build of the set when an obligation references an unknown condition
EVIDENCE_GAP	pkg/qualification/reverseproof	obtained / observed-absent / unobtainable / unattempted	never reporting "we never looked" as "it was not there"
CONTRADICTION	pkg/insurance/contradiction	conflicts within and against the set	letting an unresolved material contradiction defeat sufficiency rather than averaging it away
QUALIFICATION	pkg/qualification/state	the claim's qualification state	having no PROVEN state to reach — the type system has none
NEXT_BEST_EVIDENCE	pkg/qualification/nextbest	what is worth obtaining next	excluding a rights-denied candidate rather than scoring it low

4. Ordering is enforced, not conventional

`Execution.Complete` refuses a stage whose predecessors have not run. Two consequences are worth stating on their own:

TRUST precedes FINDING. A finding that has not passed through trust assessment is an opinion. `TestTrustPrecedesFinding` asserts the ordering; `TestReasoningCannotJumpStraightToDecision` asserts that reasoning cannot reach a decision directly — reasoning proposes.

QUALIFICATION precedes NEXT_BEST_EVIDENCE, and a reverse run that *stops* at QUALIFICATION is **incomplete**. This is deliberate and is the reverse direction's whole point: "we know what to get next" is the

successful end of a reverse run. A run that diagnoses the gap without saying what to do about it is the half-finished analysis the reverse direction exists to prevent.

5. Closure — the check that the two are one architecture

Two pipelines can share a repository without being one system. `fref.Close` tests whether they actually are, over a single subject:

- **Unrequired evidence** — the forward run's finding rests on evidence *no proof obligation called for*. That is how a system ends up "supported" by evidence nobody can say why it needed.
- **Unmet obligations** — the reverse run required something the forward run never used.

Closure holds only when both sets are empty. `Closure.Explain()` names the specific hashes on either side.

Note what `Close` does *not* do: it does not decide what counts as "the evidence a finding rests on". `pkg/proof` answers that (its `EvidenceSet`), and a second answer computed here would be exactly the duplicate authority the architecture forbids.

6. What this contract does not establish

- It has never been run against a production execution. Every `Execution` in the test suite is constructed by a test.
- Recording a stage's package is voluntary: `VerifyAgainstContract` reports drift only for records that name one. A caller that names nothing is not asserting anything, and is not flagged.
- Closure compares evidence identifiers, not evidence. Two runs can close over the same hashes and still be about different things if the hashes are wrong — which is why TECP pins them and `proof.VerifyHash` re-derives them.